Scenario 2: Bad ConfigMap Rollout

Objective

Diagnose a rollout where application configuration and Kubernetes health probes no longer agree.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/break-config.sh student-portal
```

The script saves the current port, changes the ConfigMap, and restarts the student portal backend Deployment. Wait for Datadog and Kubernetes to observe the rollout failure before troubleshooting.

Situation

The student portal has reduced rollout capacity after a configuration release. Existing pods may still serve requests, but new pods do not become Ready.

Symptoms

- New pods fail to become Ready.
- Restart counts may increase while older pods still serve.

What You Know

- Datadog shows unavailable replicas and increasing container restarts for `student-portal-backend`.
- Kubernetes health events began after a rollout.
- The process in each new container appears to start.

Start With Datadog

1. Go to **Infrastructure > Kubernetes > Explorer**, select **Pods**, set **Past 30 Minutes**, and enter `kube_namespace:student-portal kube_deployment:student-portal-backend` in **Filter by**. Compare Ready status and restarts on the old and new pods.
2. Go to **Events > Explorer** and search `kube_namespace:student-portal status:(warning OR error)`. Open the readiness-probe, liveness-probe, or BackOff event for the newest backend pod.
3. Go to **Logs > Explorer**, set **Past 30 Minutes**, and search `service:student-portal-backend`. Open a startup log and record the port on which the process is listening.

Record the newest pod, first probe-event timestamp, restart change, and logged listening port.

Troubleshooting

1. Check the rollout and identify the unhealthy new pod.

```
kubectl get deployment student-portal-backend -n student-portal
kubectl get pods -n student-portal -l app=student-portal-backend
```

Expected output: the rollout is incomplete, and the newest pod is not Ready or is restarting while older pods remain Running.

1. Inspect the new pod.

```
kubectl describe pod <pod-name> -n student-portal
```

Expected output: events show readiness or liveness probe failures against port `4000`, often followed by `BackOff` or a container restart.

1. Read the application startup logs.

```
kubectl logs <pod-name> -n student-portal
```

Expected output: the application reports that it is listening on port `4001`. This conflicts with the probes checking port `4000`.

1. Compare the ConfigMap value with the Deployment probes.

```
kubectl get configmap student-portal-backend-config -n student-portal -o yaml
kubectl get deployment student-portal-backend -n student-portal \
  -o jsonpath='{.spec.template.spec.containers[0].readinessProbe.httpGet.port}{"\n"}'
```

Expected output: the ConfigMap contains `PORT: "4001"`, but the readiness probe returns `4000`. The port mismatch is the root cause.

Important Clues

Connect the probe failures to the application listening port and the port checked by the readiness and liveness probes.

Root Cause

Record the configuration and probe mismatch supported by your evidence. Confirm it with the instructor after the exercise.

Fix

```
./scripts/chaos/break-config.sh student-portal --undo
```

The script restores the saved `PORT` value and restarts the Deployment.

Validation

```
kubectl rollout status deployment/student-portal-backend -n student-portal
kubectl get pods -n student-portal
```

Confirm every pod is Ready, restarts stop increasing, the app works, and both Datadog monitors recover.

DevOps Lesson

Validate configuration and health-probe contracts together before rollout.

STAR Method

Situation

Summarize the partial-capacity rollout.

Task

Explain why only new pods needed investigation.

Action

Describe the Datadog, event, log, ConfigMap, and probe comparison.

Result

State how readiness and monitor recovery were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

One of our internal applications stopped bringing new pods to Ready after a configuration rollout. My task was to identify why only the new pods failed. Datadog events and logs showed probe failures and an application listening on port 4001, while Kubernetes probes still checked port 4000. I restored the correct ConfigMap value and restarted the Deployment. All replicas became Ready, and configuration-to-probe validation was added to deployment checks.